WEEK-1: Installation of R-Studio on windows:

Step – 1: With R-base installed, let's move on to installing RStudio. To begin, goto download RStudioand click on the download button for RStudio desktop.

Step–2: Click on the link for the windows version of RStudio and save the.exe file.

Step–3: Run the .exe and follow the installation instructions. Click Next on the welcome window.

Enter/ browse the path to the installation folder and click Next to proceed.

Select the folder for the start menu shortcut or click on do not create shortcuts and then click Next. Wait for the installation process to complete.

Click Finish to end the installation.

Install the R Packages:-

- First, run RStudio.
- After clicking on the packages tab, click on install. The following dialog box will appear.
- In the Install Packages dialog, write the package name you want to install under the Packages field and then click install. This will install the packageyousearchedfororgiveyoualistofmatchingpackagesbasedonyour package text.

Installing Packages:

Loading Packages:- Once the package is downloaded to your computer you can access the functions and Resources provided by the package in two different ways: #load the package to use in the current R session library (package name)

Getting Help on Packages:- "C:/Program Files/R/R-3.2.2/library" install.packages("Package Name")
Install the package named "XML". install.packages("XML")

WEEK-2: Learn all the basics of R-Programming (Data types, Variables, Operators etc.)

Data Types:

```
num <- 50.25
integer_num <- 100L
name <- "Anil"
status <- TRUE
complex_num <- 6+2i
print(num)
print(integer_num)
print(name)
print(status)
print(complex_num)
```

Output:

```
[1] 50.25
[1] 100
[1] "Anil"
[1] TRUE
[1] 6+2i
```

Vectors

```
# Create a vector.
apple <- c('red','green',"yellow")
print(apple)
print(class(apple))
```

Output:

```
[1] "red"    "green"  "yellow"
[1] "character"
```

Program to swap 2 numbers

```
a <- 5
b <- 6
print("before swapping")
print(a)
print(b)
a <- a+b
b <- a-b
a <- a-b
print("after swapping")
print(a)
print(b)
```

Output:

"before swapping"

5

6

"after swapping"

6

5

Arrays

```
# Create an array.
a <- array(c('green','yellow'),dim= c(3,3,2))
print(a)
```

Output:

, , 1

[,1] [,2] [,3]

[1,] "green" "yellow" "green"

[2,] "yellow" "green" "yellow"

[3,] "green" "yellow" "green"

, , 2

     [,1]   [,2]   [,3]

[1,] "yellow" "green" "yellow"

[2,] "green" "yellow" "green"

[3,] "yellow" "green" "yellow"

Factors

```
# Create a vector.
apple_colors<- c('green','green','yellow','red','red','red','green')
# Create a factor object.
factor_apple<- factor(apple_colors)
# Print the factor.
print(factor_apple)
print(nlevels(factor_apple))
```

Output:

[1] green green yellow red   red   red   green

Levels: green red yellow

[1] 3

Variables

```
# Assignment using equal operator.
var.1=c(0,1,2,3)
# Assignment using leftward operator.
var.2<- c("learn","R")
```

```
# Assignment using rightward operator.
c(TRUE,1)->var.3
print(var.1)
cat ("var.1 is ",var.1,"\n")
cat ("var.2 is ",var.2,"\n")
cat ("var.3 is ",var.3,"\n")
```

Output:

```
[1] 0 1 2 3
var.1 is  0 1 2 3
var.2 is  learn R
var.3 is  1 1
```

R Operators

```
a <- 15
b <- 4
cat("Arithmetic Operators\n")
cat("Addition:", a + b, "\n")
cat("Subtraction:", a - b, "\n")
cat("Multiplication:", a * b, "\n")
cat("Division:", a / b, "\n")
cat("Power:", a ^ b, "\n")
cat("Modulus:", a %% b, "\n")
cat("\nRelational Operators\n")
cat(a == b, "\n")
cat(a != b, "\n")
cat(a > b, "\n")
cat(a < b, "\n")
cat(a >= b, "\n")
cat(a <= b, "\n")
```

```
cat("\nLogical Operators\n")
cat(TRUE & TRUE, "\n")
cat(TRUE | FALSE, "\n")
cat(!TRUE, "\n")
cat("\nAssignment Operators\n")
x <- 50
y <- 100
z <- 200
cat(x, y, z, "\n")
cat("\nSpecial Operators\n")
print(1:5)
print(3 %in% c(1, 2, 3, 4))
A <- matrix(c(1, 2, 3, 4), 2)
B <- matrix(c(5, 6, 7, 8), 2)
print(A %*% B)
```

Output:

```
Arithmetic Operators
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Power: 50625
Modulus: 3

Relational Operators
FALSE
TRUE
TRUE
FALSE
```

TRUE

FALSE

Logical Operators

TRUE

TRUE

FALSE

Assignment Operators

50 100 200

Special Operators

[1] 1 2 3 4 5

[1] TRUE

     [,1] [,2]

[1,]   23   31

[2,]   34   46

WEEK-3: Implement R-Loops with different examples.

For loop

```
# Create fruit vector
fruit <- c('Apple', 'Orange',"Guava", 'Pinapple')
# Create the for statement
for ( i in fruit){
print(i)
}
```

Output:

```
[1] "Apple"
[1] "Orange"
[1] "Guava"
[1] "Pinapple"
```

Repeat loop

```
i <- 1
repeat {
   print(i)
   i <- i + 1

   if (i > 3) {
      break
   }
}
```

Output:

```
[1] 1
[1] 2
[1] 3
```

R while loop

```
v <- c("Hello","while loop")
cnt <- 2
while (cnt < 7) {
print(v)
cnt = cnt + 1
}
```

Output:

```
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
```

If and break loop

```
i <- 1
repeat {
   print(i)
   i <- i + 1

   if (i > 3) {
      break
   }
}
```

Output:

```
[1] 1
[1] 2
[1] 3
```

Next loop

```
x <- 1:10
for (i in x) {
   if (i %% 2 != 0) {
      next
   }
   print(i)
}
```

Output:

```
[1] 2
[1] 4
[1] 6
[1] 8
[1] 10
```

WEEK-4: Learn the basics of functions in R and implement with examples.

Built-in Function

```
# Create a sequence of numbers from 32 to 44.
print(seq(32,44))
# Find mean of numbers from 25 to 82.
print(mean(25:82))
# Find sum of numbers frm 41 to 68.
print(sum(41:68))
```

Output:

```
[1] 32 33 34 35 36 37 38 39 40 41 42 43 44
[1] 53.5
[1] 1526
```

User-defined Function

```
# Create a function to print squares of numbers in sequence.
new.function <- function(a) {
for(i in 1:a) {
b <- i^2
print(b)
} }
new.function(6)
```

Output:

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
[1] 36
```

WEEK-5: Write R programs using Matrices, Arrays, Lists, Vectors and Factors

Vectors

```
cat("Vectors\n")
temp <- c(20.5, 32.4, 45.6, 24.3)
print(temp)
names <- c("Ramu", "Raju", "Seeta")
print(names)
```

Output:

```
Vectors
[1] 20.5 32.4 45.6 24.3
[1] "Ramu"  "Raju"  "Seeta"
```

Lists

```
cat("Lists\n")
employee <- c(101, 2.5, "Ramu")
numeric <- c(100, 10.5, 20.48)
print(employee)
print(numeric)
```

Output:

```
Lists
[1] "101"  "2.5"  "Ramu"
[1] 100.00  10.50  20.48
```

Matrices

```
# Create a matrix.
M =matrix( c('a','a','b','c','b','a'),nrow=2,ncol=3,byrow= TRUE)
print(M)
```

Output:

```
     [,1] [,2] [,3]
```

```
[1,] "a"  "a"  "b"
[2,] "c"  "b"  "a"
```

Arrays

```
cat("Array\n")
temp <- array(1:24, dim = c(4, 3, 2))
print(temp)
```

Output:

```
Array
, , 1

     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12

, , 2

     [,1] [,2] [,3]
[1,]   13   17   21
[2,]   14   18   22
[3,]   15   19   23
[4,]   16   20   24
```

Factors

```
cat("Factor\n")
education_level <- factor(
  c("highschool", "Bachelor", "Master", "Bachelor")
```

)

print(education_level)

Output:

Factor

[1] highschool Bachelor Master Bachelor

Levels: Bachelor highschool Master

WEEK-6: Write R programs using Data frames.

```
# Create vectors of equal length
names <- c("Alice", "Bob", "Charlie")
ages <- c(25, 30, 22)
is_member <- c(TRUE, FALSE, TRUE)
# Combine them into a data frame
my_df <- data.frame (Name = names, Age = ages, Member = is_member)
print(my_df)
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
print(Data_Frame)
```

Output:

```
Name Age Member
1   Alice  25   TRUE
2     Bob  30  FALSE
3 Charlie  22   TRUE
  Training Pulse Duration
1 Strength   100       60
2  Stamina   150       30
3    Other   120       45
```

WEEK-7: Write R programs to read a file into a data frame and Display it(CSV and Excel)

a) Program to read a file into a data frame

```
data <- read.csv("C:/Users/Eclass 40II/Downloads/people.csv")
print(data)
```

Output:

Person-ID	name	first	last
3130	Brooks, Rusella	Rusella	Burks
3247	Avila, Damein	Damein	Avila
3547	Olsen, Robin	Robin	Olsen
1538	Moises, Edger	Edger	Moises

b) Getting the maximum salary

```
csv-data <- read.csv("C:/Users/Eclass/F-class-md Desktops/5A5/record.csv")
max-sal <- max(csv-data$salary)
print(max-sal)
```

Output:

[1] 860

c) Getting the details of all the persons who are working in IT dept.

```
csv-data <- read.csv("C:/Users/Eclass/F-class/desktop/5A5/record.csv")
details <- subset(csv-data, dept == "IT")
print(details)
```

Output:

ID	Name	Salary	dept	start_date
1	Raju	600	IT	12-04-2012
2	Ramu	720	IT	03-06-2016

d) Getting the details of person whose salary is greater than 600 and in IT

```
csv-data <- read.csv("C:/Users/Eclass/F-class/Desktop/5A5/record.csv")
details <- subset(csv-data, dept == "IT" & salary > 600)
print(details)
```

Output:

ID	Name	Salary	dept	start_date
2	Ramu	720	IT	03-06-2024

e) Getting details of people joined after 2014

```
csv-data <- read.csv("C:/Users/Desktop/5A5/record.csv")
details <- subset(
   csv-data,
   as.Date(start_date, format = "%d-%m-%Y") >=
   as.Date("01-01-2014", format = "%d-%m-%Y")
)
print(details)
```

Output:

ID	Name	Salary	dept	start_date
2	Ramu	720	IT	03-06-2016
3	Seeta	880	HR	16-08-2014

f) Reading the text file

```
df <- read.table(
   "C:/Users/Eclass-4011/Downloads/people.txt",
   sep = "\t",
   header = TRUE
)
```

print(df)

Output:

Name	Age	Roll-Number
Akshu	21	5A5
Gayathri	25	5C9
Bhoomi	23	5H0

WEEK-8: Write R programs to write a data frame to a file and Display it(CSV and Excel)

Writing into a CSV file:

```
# Step 1: Create a sample data frame
employee_data <- data.frame (
  ID = c(101, 102, 103),
  Name = c ("Alice", "Bob", "Charlie"),
  Department = c ("HR", "IT", "Finance")
)
# Step 2: Write the data frame to a CSV file
write.csv(employee_data, file = "employees.csv", row.names = FALSE)
message("CSV file successfully written.")
# Step 3: Read the CSV file back into R
imported_csv <- read.csv(file = "employees.csv")
# Step 4: Display the content
message("\n--- Displaying Imported CSV Data ---")
print(imported_csv)
```

Output:

```
CSV file successfully written.
 --- Displaying Imported CSV Data ---
  ID      Name    Department
1 101    Alice         HR
2 102     Bob         IT
3 103  Charlie    Finance
```

Writing into a Excel file:

```
 # Step 1: Install and load required packages
if(!require("writexl")) install.packages("writexl")
if(!require("readxl")) install.packages("readxl")
library(writexl)
library(readxl)
# Step 2: Create a sample data frame
 product_data <- data.frame(
  ProductID = c(1, 2, 3),
  Item = c("Laptop", "Mouse", "Keyboard"),
  Stock = c(50, 150, 85)
)
# Step 3: Write the data frame to an Excel file
write_xlsx(product_data, path = "products.xlsx")
message("Excel file successfully written.")
# Step 4: Read the Excel file back into R
imported_excel <- read_excel(path = "products.xlsx")
# Step 5: Display the content
message("\n--- Displaying Imported Excel Data ---")
print(as.data.frame(imported_excel))
```

Output:

```
Excel file successfully written.
 --- Displaying Imported Excel Data ---

      ProductID    Item        Stock
1       1        Laptop        50
2       2        Mouse        150
3       3        Keyboard      85
```

WEEK-9: Write an R program to Connect to a database and read the data

```
# Step 1: Install and load required database packages
if(!require("DBI")) install.packages("DBI")
if(!require("RSQLite")) install.packages("RSQLite")
library(DBI)
library(RSQLite)
# Step 2: Establish connection to the database
# (Using ":memory:" creates a temporary, in-memory SQLite database)
con <- dbConnect(RSQLite::SQLite(), dbname = ":memory:")
message("Successfully connected to the database.")
# --- Setup Sample Data (For demonstration purposes) ---
sample_data <- data.frame(
  UserID = c(1, 2, 3),
  Username = c("john_doe", "jane_smith", "alex_v"),
  Status = c("Active", "Inactive", "Active")
)
dbWriteTable(con, "users", sample_data)
# Step 3: Read entire table directly
message("\n--- Method 1: Reading Whole Table ---")
users_table <- dbReadTable(con, "users")
print(users_table)
# Step 4: Read data using a custom SQL Query
message("\n--- Method 2: Reading with SQL Query ---")
sql_query <- "SELECT UserID, Username FROM users WHERE Status = 'Active'"
queried_data <- dbGetQuery(con, sql_query)
print(queried_data)
# Step 5: Always close the database connection when finished
dbDisconnect(con)
message("\nDatabase connection safely closed.")
```

Output:

Successfully connected to the database.

--- Method 1: Reading Whole Table ---

UserID Username Status

1 1 john_doe Active

2 2 jane_smith Inactive

3 3 alex_v Active

--- Method 2: Reading with SQL Query ---

UserID Username

1 1 john_doe

2 3 alex_v

Database connection safely closed.

10. Implement a) Linear regression model and b) Logistic regression model of any data set and print confusion matrix, precision recall, F1 score, AUC curve.

```
# Step 1: Install and load required evaluation packages
if(!require("caret")) install.packages("caret", dependencies=TRUE)
if(!require("pROC")) install.packages("pROC")
library(caret)
library(pROC)
data(mtcars)
message("=== [A] LINEAR REGRESSION MODEL ===")
linear_model <- lm(mpg ~ hp + wt, data = mtcars)
linear_summary <- summary(linear_model)
cat("R-squared: ", linear_summary$r.squared, "\n")
cat("Residual Standard Error (RMSE): ", linear_summary$sigma, "\n\n")
message("=== [B] LOGISTIC REGRESSION MODEL ===")
logistic_model <- glm(am ~ mpg + hp, data = mtcars, family = binomial)
prob_predictions <- predict(logistic_model, type = "response")
class_predictions <- factor(ifelse(prob_predictions > 0.5, 1, 0), levels = c("0", "1"))
actual_values <- factor(mtcars$am, levels = c("0", "1"))

# 1. Generate and Print Confusion Matrix
conf_matrix <- confusionMatrix(class_predictions, actual_values, positive = "1")
print(conf_matrix$table)

# 2. Extract and Print Precision, Recall, and F1-Score
precision_val <- precision(class_predictions, actual_values, positive = "1")
recall_val    <- recall(class_predictions, actual_values, positive = "1")
f1_val        <- F1_val <- (2 * precision_val * recall_val) / (precision_val + recall_val)
cat("\n--- Classification Performance Metrics ---\n")
cat("Precision : ", precision_val, "\n")
cat("Recall    : ", recall_val, "\n")
```

```
cat("F1 Score  : ", f1_val, "\n")

# 3. Compute and Plot AUC / ROC Curve
roc_curve <- roc(actual_values, prob_predictions, quiet = TRUE)
cat("AUC Value : ", auc(roc_curve), "\n")

# Display the ROC Curve plot window
plot(roc_curve, col = "blue", main = "ROC Curve - Logistic Regression Model",
    xlab = "False Positive Rate (1 - Specificity)", ylab = "True Positive Rate (Sensitivity)")
abline(a = 0, b = 1, lty = 2, col = "darkgray")
```

Output:

```
=== [A] LINEAR REGRESSION MODEL ===
R-squared:  0.8267855
Residual Standard Error (RMSE):  2.568411

=== [B] LOGISTIC REGRESSION MODEL ===
          Reference
Prediction  0  1
         0 16  3
         1  3 10
 --- Classification Performance Metrics ---
Precision :  0.7692308
Recall    :  0.7692308
F1 Score  :  0.7692308
AUC Value :  0.8990385
```

11. Write R script to plot Bar Chart, Pie Chart, Histogram, Box plot, Dot plot

Bar Plot or Bar Chart

```
x <- c(7, 15, 23, 12, 44, 56, 32)
png(file = "barplot.png")
barplot(x, xlab = "Malla Reddy Students",
      ylab = "Count", col = "white",
      col.axis = "darkgreen",
      col.lab = "darkgreen")
dev.off()
```

Pie Diagram or Pie Chart

```
x <- c(210, 450, 250, 100, 50, 90)
names(x) <- c("Algo", "DS", "Java", "C", "C++", "Python")
png(file = "piechart.png")
pie(x, labels = names(x), col = "white",
main = "Articles on GeeksforGeeks", radius = -1,
col.main = "darkgreen")
dev.off()
```

Histogram

```
x <- c(21, 23, 56, 90, 20, 7, 94, 12,
   57, 76, 69, 45, 34, 32, 49, 55, 57)
png(file = "hist.png")
      xlab = "Values",
      col.lab = "darkgreen",
      col.main = "darkgreen")
dev.off()
```

Box Plot

```
x <- c(42, 21, 22, 24, 25, 30, 29, 22,
   23, 23, 24, 28, 32, 45, 39, 40)
png(file = "boxplot.png")
boxplot(x, xlab = "Box Plot", ylab = "Age",
col.axis = "darkgreen", col.lab = "darkgreen")
dev.off()
```

12. Use ggplot2 and implement 1) Univariate, 2) Bivariate graphs

1. Univariate Graphs

Histogram (Continuous)

```
ggplot(data = mtcars, aes(x = hp)) +
  geom_histogram(binwidth = 5) +
  labs(title = "Histogram of Horsepower",
       x = "Horsepower",
       y = "Count")
```

Bar Chart (Categorical)

```
library(ggplot2)
df <- data.frame(
  id = 1:100,
  category = sample(c("A", "B", "C"), 100, replace = TRUE)
)
ggplot(df, aes(x = category)) +
  geom_bar(fill = "steelblue", color = "black") +
  theme_minimal() +
  labs(
    title = "Univariate Bar Chart",
    x = "Category",
    y = "Count"
  )
```

2) Bivariate graph

Scatter plot

```
library(ggplot2)
ggplot(data = mtcars, aes(x = hp, y = mpg)) +
  geom_point() +
```

```
 labs(
   title = "Horsepower vs. Miles Per Gallon",
   x = "Horsepower (hp)",
   y = "Miles Per Gallon (mpg)"
 )
```

Grouped Bar Plots

```
library(ggplot2)
cities <- c(rep("Delhi", 3), rep("Mumbai", 3), rep("Chennai", 3), rep("Bengaluru", 3))
humidity <- rep(c("High", "Medium", "Low"), 4)
temperature <- abs(rnorm(12, 25, 10))
dataframe <- data.frame(cities, humidity,  temperature)
dataframe
ggplot(dataframe, aes(fill = humidity,  y = temperature, x = cities))
geom_bar(position = "dodge", stat = "identity")
ggtitle("Weather Data of 4 Cities !")
theme(plot.title = element_text(hjust = 0.5))
```